

Downshift Statham Product Discovery Report

1. What This Is

A small product discovery app for a 4,000 item home goods catalog. The goal was not to build a full store; the goal was to inspect the data, make practical search decisions, and create a browsing experience that feels useful.

The project started from a free open source Next.js ecommerce template with Shopify-style structure and visual direction. I kept the useful Next.js/Tailwind/project scaffolding, then replaced the core behavior with local JSON catalog loading, local search/ranking, local product pages, local cart state, scoped filters, image fallbacks, and discovery sections. There is no Shopify backend and no checkout flow.

2. Build Shape

```
Provided items.json
  |
  v
Local public/data/items.json
  |
  v
normalizeProducts()
  |
  +--> searchProducts()
  |   |
  |   v
  |   ranked results grid
  |
  +--> getFacetOptions()
  |   |
  |   v
  |   category / brand / material chips
  |
  +--> curated sections
  |   |
  |   v
  |   recent launches / best rated / price bands
```

Implemented surfaces:

- / search and discovery homepage
- /products/[product] local product detail page
- /cart local cart page
- /data/items.json local catalog file

Supporting behavior:

- animated fallback for missing or broken product images
- localStorage cart with add, quantity update, remove, clear, subtotal, and count
- no checkout, because the task is product discovery
- build warning cleanup for deprecated generated `color-adjust` CSS

3. Data Probe Summary

Catalog

- 4,000 products
- 10 categories
- 16 brands
- 83 unique tag values

Main columns

- id, title, brand, category, tags, price, rating, reviews,
- inStock, releasedAt, image, imageWidth, imageHeight, description

Data issues handled

- null prices
- string prices with commas
- missing ratings / reviews
- missing images
- missing descriptions
- inconsistent title casing / spacing
- future release dates

Search-relevant finding:

Product titles generally look like:

style / material / product noun

Examples:

- Brushed Rattan Crate
- Minimal Linen Planter Box
- Vintage Brass Throw Blanket

Useful search dimensions:

- product type
- material
- style
- room / category
- brand
- price intent
- rating intent

One caveat: tags are not pure materials. They include category-like values, materials, styles, and product-type terms. The UI labels the row as "Material" because that is the most useful compact browsing label, but technically it is a tag-backed material/style row.

4. Product Normalization Flow

```

raw product
|
v
normalizeProduct()
|
+--> title
|   compact whitespace
|   title-case only when all caps / all lowercase
|
+--> brand / category
|   compact strings
|   fallback labels if missing
|
+--> tags
|   ensure array
|   compact each value
|
+--> price
|   number stays number
|   "1,081.43" -> 1081.43
|   invalid / empty -> null
|
+--> rating
|   only kept when reviews > 0
|
+--> available
|   inStock must be true
|   releasedAt must not be in the future

```

Why this matters: search and display should not leak raw catalog messiness into the user experience. A product with a future release date can be in the data but should not look immediately available.

5. Filter Scoping Flow

```

All products
|
v
Category chips
|
+-- selected category = All
|   |
|   v
|   brands from all products
|
+-- selected category = Bath
|   |
|   v
|   brands only from Bath products
|   |
|   v
|   material/tag chips only from Bath + selected brand

```

The chip rows are deliberately scoped so lower-level choices remain meaningful. If a category is selected, brands that have no products in that category disappear. If a brand is selected, material/style chips with no matching products disappear.

6. Search Flow

```
User input
  query + category + brand + material/tag
    |
    v
Hard filters first
  category exact match unless All
  brand exact match unless All
  selected tag must be in product.tags unless All
    |
    v
Tokenize query
  lowercase
  split by whitespace
    |
    v
Detect shopping intent
  cheap / budget / affordable -> low price
  premium / luxury / expensive -> premium
  best / top / popular -> rating/review quality
    |
    v
Remove intent words from required text terms
  |
  v
Score remaining products
  |
  v
Sort and render
```

Example:

```
"cheap rattan chair"
  |
  v
intent: low price
required terms: rattan, chair
  |
  v
product must match both rattan and chair somewhere
then cheaper matching products are boosted
```

Intent-only searches are allowed:

```
"cheap"
  -> rank the currently filtered catalog by low-price intent

"best"
  -> rank the currently filtered catalog by rating/review strength
```

7. Text Matching And Ranking

Searchable fields:

```
title
category
tags
brand
description
```

Required term rule:

For query terms after removing intent words:

every term must match at least one searchable field

Example:

"rattan chair"

rattan must match somewhere

chair must match somewhere

Scoring priority:

Phrase match

title strongest

category

tags

brand

description weakest

Individual term match

title strongest

category

tags

brand

description weakest

After text score:

score

+ availability boost

+ image boost

- missing image penalty

- missing price penalty

+ rating/review quality

+ price/rating intent boost when query asks for it

Final sort:

1. score descending
2. products with images first
3. available products first
4. title alphabetically

This keeps search inspectable. The app is not pretending to be semantic search; it is deterministic local ranking tuned to the shape of this catalog.

8. Homepage Rendering Flow

```

No active search/filter
  |
  v
Discovery mode
  recent launches carousel
  6 product cards
  best-rated carousel
  6 product cards
  best-rated under $5,000 carousel
  6 product cards
  best-rated under $10,000 carousel
  remaining cards

Any active search/filter
  |
  v
Focused results mode
  hide carousels
  render matching product grid

```

The split keeps the default page browseable and editorial, while search/filter actions switch the UI into a simpler results-first mode.

9. Cart And Product Pages

```

Product card
  |
  +--> click card content
  |   |
  |   v
  |   /products/[id]
  |
  +--> Add to cart
  |   |
  |   v
  |   CartProvider
  |   |
  |   v
  |   localStorage
  |   |
  |   v
  |   /cart

```

The cart is intentionally local and lightweight. It supports browsing behavior without adding payment or checkout complexity that was outside the assignment.

10. Validation

```

npm run build
passed

Production route checks
/                200
/cart            200
/products/1      200
/data/items.json 200

Data sanity
4,000 products
4,000 unique IDs
normalized rows usable by the app

```

Limitation: the template does not currently include Playwright, Cypress, or another browser automation runner, so validation was build and route oriented rather than a full automated visual/click suite.

11. Tradeoff To Watch

Current choice

client-side deterministic search

Why it fits

4,000 products is small
ranking is easy to inspect
no backend needed for the task

When it stops fitting

much larger catalog
typo tolerance
synonyms
semantic matching
analytics-based ranking
personalization

Next move

move ranking into an indexed search service
keep this scoring model as the relevance spec

The next product improvement would be to split `tags` into cleaner facets: material, style, product type, and room. That would make the filter UI more accurate without changing the basic search model.